

What we did, what broke, and what it taught us.

A running log of the build. Every entry records what was attempted, what went wrong, what we changed in response, and the lesson worth keeping. The failures are the useful part: several of them are the reason the final system is defensible, and at least one of them is the strongest thing we can put in front of a reviewer.

HOW TO READ THIS

WORKED	did what it was supposed to, first time
FIXED	broke, was diagnosed, and now works
FAILED	did not work and was abandoned
FALSIFIED	a hypothesis we tested and disproved
SHIPPED	in the running application

Phase 1 — Understanding what we already had

01 Audit of the existing AREE codebase

WORKED

WHAT WE DID

Cloned github.com/jeyamoorthi/urbanlive-ai and read all 4,533 lines of backend Python before writing anything new, rather than trusting the pitch deck's description of the system.

LESSON

Two findings changed the plan. backend/ingestion/weather_stream.py was 0 bytes — the system ingested no meteorology at all — and the 'forecast' was `numpy.polyfit` degree 1 on recent AQI, projected 5 to 30 minutes. For a problem statement about weather-chemistry coupling over 72 hours, the gap was much larger than the architecture assumed. Read the code, not the deck.

02 Designing λ , the feedback loop gain

WORKED

WHAT WE DID

Formulated the aerosol-radiation-PBL feedback as a product of three log-elasticities with closed-loop amplification $1/(1-\lambda)$, the same algebra used for climate feedback factors.

LESSON

A differentiator has to come out of the problem statement's own words, not be bolted on. The PS explicitly says these feedback loops are ignored; λ was an attempt to measure exactly the thing it named.

03 Synthetic self-test before touching real data

WORKED

WHAT WE DID

Built a generator with a prescribed λ and deliberately oversized diurnal cycles, then checked the estimator recovered it. It did: true 0.1386, recovered 0.1513, 9.2% error, all three sign checks passing.

LESSON

This passed, and it still misled us. A self-test can only falsify an estimator — it can never confirm that real data contains what you hope. It validated the ARITHMETIC, not the premise. Worth saying out loud to a reviewer: it is the difference between a team that tested its method and one that believed it.

Phase 2 — Getting the data

04 ERA5 from the Copernicus CDS

FIXED

WHAT WE DID

Wrote a CDS API client for 23 monthly ERA5 requests covering five winters of NCR meteorology.

WHAT WENT WRONG

No Copernicus key was available, and CDS requests queue for minutes to hours even with one. The single most important variable — boundary layer height — was behind that gate, and the whole project was blocked on it.

WHAT WE DID ABOUT IT

Found that Open-Meteo's archive API re-serves ERA5 over plain REST with no key, no queue, and an arbitrary date range in one call — including `boundary_layer_height` and, crucially, `terrestrial_radiation` for the clearness index. Four and a half years of hourly data for a 9-point NCR grid arrived in 90 seconds.

LESSON

Before accepting that a project is blocked on a credential, check whether the same data is re-served somewhere without one. The CDS client stayed in the repo as the canonical path; the REST path is what actually unblocked the work.

05 OpenAQ sensor selection — three separate failures

FIXED

WHAT WE DID

Pulled hourly ground PM2.5 for reference-grade NCR stations across 2020–2025.

WHAT WENT WRONG

Three distinct bugs, each of which returned plausible-looking but wrong data rather than erroring. **(a)** A location's date range covers every sensor ever sited there, while individual sensor ids retire — selecting on location dates returned zero rows for recent windows. **(b)** Yearly request chunks forced deep pagination and OpenAQ answered with 500s and 408s, silently losing three stations. **(c)** Deduplicating by location kept the longest-overlap sensor, which is the OLD generation — this truncated the record at Oct 2022 and quietly removed both holdout Novembers.

WHAT WE DID ABOUT IT

Probed every sensor for its own coverage and cached the result; switched to monthly chunks so one request always fits inside a single 1000-row page; stopped deduplicating across sensor generations and added the continuously-operating US Embassy monitor as the anchor that spans OpenAQ's Nov 2022 – Feb 2025 gap in the Indian feed.

LESSON

Bug (c) is the dangerous kind. Nothing failed. The pipeline ran, produced a clean-looking dataset, and had silently deleted the evaluation period. Always print the coverage of what you actually loaded, not just its row count — the panel builder now reports stations per season every run.

06 CPCB via data.gov.in

FIXED

WHAT WE DID

Wrote a connector for the authoritative CPCB CAAQMS feed to replace WAQI, which gives neither real station names nor a usable observation timestamp.

WHAT WENT WRONG

The endpoint answered a single request in 2–4 seconds but returned HTTP 502 with a zero-length body after ~60 seconds under a burst — never a 429, so ordinary rate-limit handling did not catch it.

WHAT WE DID ABOUT IT

Added backoff that treats a 5xx with an empty body as rate limiting.

LESSON

Read the failure mode, not the documentation. An API that rate-limits by timing out instead of by status code will defeat any retry logic written from the spec.

Phase 3 — The hypothesis meets the data

07 λ on five winters of real data

FALSIFIED

WHAT WE DID

Ran the full pipeline: 39,432 hours of meteorology, 29,953 hours of PM2.5, 161 labelled episodes of which 39 locked in.

WHAT WENT WRONG

λ does not separate locked-in from ventilated episodes. AUC 0.504 — chance.

WHAT WE DID ABOUT IT

Withdrew the claim rather than tuning until something crossed a threshold. Then spent the effort on diagnosing why instead.

LESSON

The validation gate existed precisely to produce this answer in week one. It cost one day. The alternative was five people building a presentation around a number that would have collapsed under the first question from an NCMRWF reviewer.

08 Diagnosing the failure

WORKED

WHAT WE DID

Tested three candidate explanations rather than assuming one: measurement error, spatial mismatch, and wrong timescale.

LESSON

The raw physics is textbook — PM2.5 falls 163 to 79 $\mu\text{g}/\text{m}^3$ as BLH rises 44 to 1102 m. What is unmeasurable is the residual after the diurnal cycle is removed: reverse regression bounds the reliability ratio of ERA5 BLH anomalies at **0.016**, roughly 98% noise. And e1 flips sign between specifications (−0.043 pooled, +0.061 dense-network, +0.084 daily). The elasticity is not small because the physics is weak; it is small because the instrument is.

09 Reformulating to avoid PBL height entirely

FAILED

WHAT WE DID

If ERA5 BLH is the problem, remove it. Defined a ventilation index from observed PM2.5 alone: the pre-dawn to afternoon drop, which is the atmosphere ventilating, measured by a reference-grade monitor.

WHAT WENT WRONG

Also fails to predict lock-in: AUC 0.554. It behaves correctly as a ventilation measure — +0.21 with daytime max BLH, +0.15 with wind — it just does not forecast which episodes persist.

WHAT WE DID ABOUT IT

Stopped trying variants. Four predictors had now been tested on the same 161 episodes, and each further attempt inflates the chance of finding something spurious.

LESSON

Knowing when to stop is part of the method. Two failures with different mechanisms pointing the same way is evidence about the problem, not an invitation to try a fifth index.

Phase 4 — The finding that rescued the project

10 Asking whether lock-in is predictable at all

WORKED

WHAT WE DID

Instead of another predictor, asked a structural question: is lock-in determined by the state at onset, or by what happens afterwards? Compared identical predictors read before and after onset.

LESSON

The answer was unambiguous. Ventilation BEFORE onset: AUC 0.514, no skill. AFTER onset: 0.736, skilful. **Lock-in is not a diagnosable regime — it is a forecastable outcome.** Two failed hypotheses became one measured finding, and it points straight at what the problem statement actually asked for: coupled forecasting.

11 Testing whether forecast skill survives

WORKED

WHAT WE DID

The 0.736 assumed perfect knowledge of the future. Used Open-Meteo's previous-runs archive — the value actually forecast for each hour one and two days earlier — to test how much survives with only a forecast.

WHAT WENT WRONG

The archive carries no boundary layer height at any lead, so the strongest predictor could not be backtested. Only wind was available, and only from Jan 2024, giving 39 episodes with 9 locked-in.

WHAT WE DID ABOUT IT

Ran the paired test on wind: analysis AUC 0.678, 1-day forecast 0.794, 2-day 0.811, with forecast-vs-analysis agreement $r = 0.75$ and RMSE 0.79 m/s.

LESSON

Forecast error does not break the chain — but the forecast 'beating' perfect knowledge is not a real effect. With 9 locked-in episodes the AUC standard error is about 0.10. **Our own script printed '166% of the ceiling'; that number is an artefact and must never appear on a slide.**

Phase 5 — Building the application

12 Calibrating the shipped decision threshold

SHIPPED

WHAT WE DID

Derived the operating point from the same episodes and the same holdout as every other result, and emitted the whole ROC curve plus three named points instead of one.

LESSON

The exchange rate between a missed episode and a false GRAP invocation is a policy judgement for CAQM, not a modelling choice. The system ships a default of 466 m²/s (hit 0.61, false alarm 0.19) and lets the operator select a different point — and states the caveat that the sample is small.

13 The forecast-to-escalation chain

SHIPPED

WHAT WE DID

Three modules: a 72-hour ventilation forecast with sustained-collapse detection, a decision layer that opens an escalation case, and four API endpoints. Verified live: state IMMINENT, collapse in 11.4 hours, intervention window 11.4 hours, case AWAITING_APPROVAL.

LESSON

The trigger is a CONJUNCTION — observed PM2.5 above the episode threshold AND a forecast ventilation collapse — because the research showed neither is sufficient alone. **Verified by the negative case: PM2.5 = 40 produces no case at all. A trigger you have not tested in its OFF state is not a trigger, it is an alarm.**

14 Keeping the forecast layer free of Pathway

SHIPPED

WHAT WE DID

Pathway ships Linux/macOS wheels only, so every existing route degrades to engine_unavailable on Windows. The new routes were built without that dependency.

LESSON

The forecast layer is upstream of the streaming layer in the data flow, so it should not be downstream of it in the dependency graph. That reasoning — rather than convenience — is why the PS 26082 deliverable runs on the machines the team develops on.

Phase 6 — Making it real, and what that exposed

15 Wiring live ground observations into the escalation

FIXED

WHAT WE DID

The forecast half of the trigger was real, but the observed half was a hand-typed query parameter. Added a module reading the live CPCB/DPCC network so the conjunction runs on measured air.

WHAT WENT WRONG

The first implementation queried OpenAQ's /parameters/2/latest with a bbox for the NCR domain and reported 198 stations. That endpoint SILENTLY IGNORES both bbox and coordinates+radius: the rows came from South Korea, Lithuania and China. The "Delhi composite" was a global median.

WHAT WE DID ABOUT IT

Switched to /v3/locations?bbox= which does honour the box, then read each active location individually. Every returned coordinate is now re-checked against the NCR box as a hard filter.

LESSON

It looked entirely plausible — a sensible station count and a sensible concentration — which is exactly what made it dangerous. **An API that ignores a filter rather than rejecting it hands you a confident wrong answer. Verify the data you got back, not the query you sent.** It was only caught because someone asked to see the individual stations.

16 Two station counters disagreeing on one screen

FIXED

WHAT WE DID

The page showed a live forecast and an observation panel reporting 33 stations.

WHAT WENT WRONG

The global status strip sat directly above it reporting 0 / 0 stations and ENGINE NOT LOADED, because it reads the Pathway engine. Two contradictory numbers on one screen read as fabrication.

WHAT WE DID ABOUT IT

Made the header and status strip report the subsystem the CURRENT view actually depends on, and added a panel naming both data sources explicitly: the forecast uses a weather model and no stations at all; the observation side uses the ground network.

LESSON

Chrome that reports a different subsystem than the page is worse than no chrome. If a number on screen contradicts another number on the same screen, users are right to distrust both — and they should.

17 Running Pathway under WSL

FAILED

WHAT WE DID

Attempted to start the Ubuntu-22.04 distro so the streaming engine could run and restore the other three tabs.

WHAT WENT WRONG

The distro will not mount: E_ACCESSDENIED on its vhdx. Isolated to that disk specifically — the docker-desktop distro launches fine. The vhdx sits at D:/WSL/Ubuntu after being moved off the default location, and its ACL is missing the generic NT VIRTUAL MACHINE/Virtual Machines entry that WSL needs after a move. Fixing it requires an elevated shell, which the working session did not have.

WHAT WE DID ABOUT IT

Stopped rather than attempting to escalate privileges, and handed over the exact `icacls` command. Then solved the underlying problem a different way — see the next entry.

LESSON

A blocker that needs someone else's authority is a handover, not a puzzle to force. But being blocked on the stated approach is not the same as being blocked on the goal.

18 Direct-mode engine: all four tabs without Pathway

SHIPPED

WHAT WE DID

Rather than wait on WSL, checked which modules actually need the Pathway runtime. Only `rag/advisory_engine.py` imports it. The GRAP state machine, persistence tracking, causal attribution, satellite fire intelligence and the report generator are all pure Python.

WHAT WE DID ABOUT IT

Built `fallback_engine.py`: samples the live CPCB network and drives the SAME state machine, exposing the same module attributes the API layer already reads. `engine.load_engine()` falls back to it when the Pathway import fails. Every state carries `mode="direct"`, and the subsystems it genuinely cannot provide (policy RAG, event-time windows, carbon tracking) are reported as unavailable rather than faked. Result: 52 stations live, real GRAP stages, all four tabs working.

LESSON

The dependency was never as deep as the failure made it look. Three tabs were dark because of one import in one module, and displaying a station's air quality never needed a streaming runtime — it needed observations and a state machine. Before accepting that a platform constraint blocks a feature, check how much of the feature actually touches the constraint.

What we would tell another team

- **Build the gate before the pitch.** A one-week experiment that can kill the central idea is worth more than a month of building on it. Ours killed it, and the project is stronger for it.
- **A passing self-test is not evidence about the world.** It tells you the code computes what you wrote down. Whether reality contains that quantity is a separate question and needs separate data.
- **Print what you loaded, not just how much.** The bug that silently deleted both holdout months did not raise an error. It produced a clean dataset of the wrong period.
- **Negative results are presentable.** 'We measured whether lock-in can be diagnosed from the present state; it cannot' is a stronger sentence than any hypothesis we could have asserted, because it is the one a reviewer cannot knock down.
- **Test the trigger in its off state.** Most demos only show the alarm firing.
- **Verify the data you got back, not the query you sent.** An API that ignores a filter instead of rejecting it will hand you a plausible, confident, wrong answer.
- **Check how much of the feature actually touches the blocker.** Three tabs were dark because of one import in one module.

Every number in this journal is reproducible from the scripts in `research/ps26082` on branch `ps26082-coupled-forecast`.

TEAM DEVENGERS · PS 26082 · JOURNAL REVISION A